FOR JAVA DEVELOPERS

One request. Two payments.

A Spring Boot API that charges per request with HTTP 402 and MPP, a Java client that pays on the Tempo test network, an AI assistant calling it over MCP, and the two fixes that stop it overpaying and paying twice

> MCP calls the tool. MPP pays the API. Java holds the limits.

A courier API answers 402 Payment Required with a challenge for 0.05 pathUSD. A Java client decides, pays on Tempo testnet, asks again with proof and checks the receipt. Without a policy it pays 5.00 when the price is raised. Without a durable record it pays twice when a response is lost.

At the time of writing, October 2026: MPP is an Internet-Draft by Stripe and Tempo, not a standard; mpp-java 0.1.6 is experimental; Tempo Moderato testnet is chain 42431 and pathUSD has 6 decimals. Testnet tokens only. No real money moves anywhere on this sheet.

This sheet covers

- The 402 flow: WWW-Authenticate: Payment, Authorization: Payment, Payment-Receipt
- MCP versus MPP, and why they stay separate here
- MppPaymentClient: parse, decide, sign, retry, check the receipt
- Paying on Tempo: the challenge-bound memo and estimated gas
- A payment policy that refuses before signing
- Settlement, redemption and recovery

Not covered here

- Production: plain HTTP on localhost; the spec requires TLS
- More than one instance: the budget lock and replay store are per process
- Pull mode, and the draft that carries MPP inside MCP
- Real money: every payment is a testnet token

Code copied from the repository at commit cd2e4d1; outputs from evidence/.

01 What each run did to the courier’s balance

Safe client, one paid quote	`16.15 to 16.2: +0.05`
Unsafe client, price raised to 5.00	`16.45 to 21.45: +5.00`
Safe client, price raised to 5.00	`REFUSED_BY_POLICY, balance unchanged`
Unsafe client, one paid response lost	`21.45 to 21.55: +0.10, two transfers`
Safe client, one paid response lost	`21.55 to 21.6: +0.05, one transfer`
Daily budget 0.25, six 0.05 requests	`Five paid, the sixth refused at 0.30`
Twenty approvals at once (test)	`Exactly five approved, 0.25 spent`

Courier balances read with bin/balance.sh before and after each run, Tempo Moderato testnet, 2026-10-09, from evidence/02, 04 and 05. The payer also pays a network fee in pathUSD of about 0.00004 per transfer, which the policy does not count.

02 The 402 flow

The client asks. The server answers 402 with WWW-Authenticate: Payment and a challenge. The request parameter is base64url JSON: amount in base units (50000, which is 0.05 with 6 decimals), currency (the pathUSD token), recipient (the courier's wallet) and chain. A 402 is a challenge, not a charge: nothing is paid yet, and the client may say no.

If the client pays, it asks again with Authorization: Payment, a credential echoing the challenge and carrying the transaction hash. The server verifies and answers 200 with a Payment-Receipt whose reference is that hash.

The spec says challenges must not be issued over plain HTTP. This demo runs on localhost as a deliberate local shortcut; anything beyond your machine needs TLS.

evidence/01-http-exchange.txt

```
== 1. Ask for the premium quote with no payment ==
> GET /quotes/premium?from=London&to=Manchester&weightKg=2
> Idempotency-Key: transcript-1791555248734

< HTTP/1.1 402
< cache-control: no-store
< content-type: application/problem+json
< www-authenticate: Payment id="XkSC0rcJNEN3rlGCVFV0lFNaERKlqaOitEO-U6YObOk",
realm="localhost:18196", method="tempo", intent="charge",
request="eyJhbW91bnQiOiI1MDAwMCIsImN1cnJlbmN5IjoiMHgyMGMwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMD
AwMDAwMDAwIiwibWV0aG9kRGV0YWlscyI6eyJjaGFpbklkIjo0MjQzMX0sInJlY2lwaWVudCI6IjB4ZTYxNGU0M2U0ZmJmO
Tg2N2JkZjRlMmRkYmQzZjYyODY1OGI5M2RkYiJ9", expires="2026-10-09T14:19:08.773192Z",
description="Premium courier quotation"

{"type":"https://paymentauth.org/problems/payment-required","title":"Payment
Required","status":402,"detail":"Payment is required for \"localhost:18196\" (Premium courier
quotation).","challengeId":"XkSC0rcJNEN3rlGCVFV0lFNaERKlqaOitEO-U6YObOk"}

decoded request =
{"amount":"50000","currency":"0x20c0000000000000000000000000000000000000","methodDetails":
{"chainId":42431},"recipient":"0xe614e43e4fbf9867bdf4e2ddbd3f628658b93ddb"}
```

evidence/01-http-exchange.txt

```
== 3. Pay: transferWithMemo, memo bound to realm and challenge ==
memo = 0xef1ed7120131de7975921eda9276c400000000000000000000047550ec0c75806
tx hash, known before broadcast =
0x2ec458ed110b51c5f38900e8cb03bb53575110c8860fd0a612bf62a74fabb76a
mined:
https://explore.testnet.tempo.xyz/tx/0x2ec458ed110b51c5f38900e8cb03bb53575110c8860fd0a612bf62a7
4fabb76a
```

evidence/01-http-exchange.txt

```
< HTTP/1.1 200
< cache-control: private
< content-type: application/json
< payment-receipt:
eyJtZXRob2QiOiJ0ZW1wbyIsInJlZmVyZW5jZSI6IjB4MmVjNDU4ZWQxMTBiNTFjNWYzODkwMGU4Y2IwM2JiNTM1NzUxMTB
jODg2MGZkMGE2MTJiZjYyYTc0ZmFiYjc2YSIsInN0YXR1cyI6InN1Y2Nlc3MiLCJ0aW1lc3RhbXAiOiIyMDI2LTEwLTA5VD
E0OjE0OjEzLjE0NzUzNFoifQ

{"courier":"Swift Courier Premium","price":8.60,"currency":"GBP","days":1}

decoded receipt =
{"method":"tempo","reference":"0x2ec458ed110b51c5f38900e8cb03bb53575110c8860fd0a612bf62a74fabb7
6a","status":"success","timestamp":"2026-10-09T14:14:13.147534Z"}
```

> One real exchange: 402 with a challenge for 50000 base units, a transfer with a challenge-bound memo, 200 with a receipt referencing the same hash.
>
> Recorded on Tempo Moderato testnet, 2026-10-09: evidence/01-http-exchange.txt and 06-replay-rejected.txt, written by HttpExchangeTranscriptTestnetIT. Test tokens only.

> Servers must not issue Payment challenges over unencrypted HTTP.
>
> MPP core draft (draft-httpauth-payment), paymentauth.org, read 2026-10-09.

03 MCP calls the tool. MPP pays the API.

MCP, the Model Context Protocol, is how an AI assistant discovers and calls tools. It is not a payment protocol. MPP, the Machine Payments Protocol, is how one program pays another over HTTP. It needs no AI: the client and the tests pay with no model anywhere.

Together: the assistant calls premium_delivery_quote over MCP; the tool calls the paid courier API using MPP. The wallet and the policy stay inside the Java application.

A separate draft, draft-payment-transport-mcp-00, carries MPP payments inside MCP messages. It is NOT used here: no Java implementation exists at the time of writing, it would need the MCP host to understand payments, and keeping them apart shows which protocol does which job.

CourierAssistant.java (the paid tool)

```
    @McpTool(name = "premium_delivery_quote",
            description = "COSTS MONEY. Buys the premium courier's next-day quote. "
                    + "The courier charges a fee per request (0.05 pathUSD on Tempo "
                    + "testnet), paid automatically only if the payment policy allows "
                    + "it: per-request limit, daily budget and approved couriers. "
                    + "Returns the quote, the fee paid and the transaction, or the "
                    + "policy's reason for refusing.")
    public PremiumQuoteResult premiumDeliveryQuote(
            @McpToolParam(description = "Collection town, e.g. London") String from,
            @McpToolParam(description = "Delivery town, e.g. Manchester") String to,
            @McpToolParam(description = "Parcel weight in kg, 0.1 to 30")
            BigDecimal weightKg) {
        return quotations.premium(new QuoteRequest(from, to, weightKg));
    }
```

.mcp.json (whole file)

```
{
  "mcpServers": {
    "courier-assistant": {
      "type": "http",
      "url": "http://localhost:${ASSISTANT_PORT:-8080}/mcp"
    }
  }
}
```

> Claude Code, restricted to the three tools, called standard_delivery_quotes, premium_delivery_quote and payment_audit; the premium quote cost 0.05 pathUSD, one transaction.
>
> A real Claude Code session restricted to this project's three MCP tools: evidence/agent-claude-code.txt, produced by bin/evidence/agent-claude-code.sh.

04 MppPaymentClient.get

Ask. If the answer is not 402, there is nothing to pay. Parse the challenge into a ChargeOffer. Ask the policy; a refusal throws before anything is signed. Sign the transfer, write it down, and only then pay. From there the client only ever resumes.

MppPaymentClient.java

```
/** requestKey names the logical request: the same key is never paid for twice. */
public PaidResponse get(URI uri, String requestKey) {
    Optional<PaymentOperation> earlier = operations.find(requestKey);
    if (earlier.isPresent()) {
        return resume(uri, earlier.get());
    }

    HttpReply first = http.get(uri, null, idempotencyKey(requestKey));
    if (!first.isPaymentRequired()) {
        return PaidResponse.free(first.body());
    }

    Challenge challenge = TempoChargePayer.challengeFrom(first);
    PolicyDecision decision = policy.approve(ChargeOffer.from(challenge));
    if (!decision.approved()) {
        throw new PaymentRefusedException(decision.reason());
    }

    SignedTransfer transfer = payer.sign(challenge);
    // Written down before any money moves. From here on we only ever resume.
    return resume(uri, operations.startPayment(requestKey, challenge, transfer));
}
```

05 Paying on Tempo

mpp-java 0.1.6 rejects a Tempo payment unless it is a transferWithMemo whose memo binds it to the server's realm and the challenge id. At the time of writing the Tempo charge draft does not say so. A random transfer of 0.05 to the courier is not enough.

Push mode: the client broadcasts its own transfer and sends the hash (type=hash). The first transfer from a brand new wallet cost about 545,000 gas because account creation is charged, so a fixed gas limit failed. Estimate every time and add a margin.

TempoChargePayer.java

```
/**
 * Every MPP SDK rejects a Tempo payment unless its memo is bound to this server
 * (realm) and this challenge (id), so nobody can claim someone else's transfer.
 */
public static String memoFor(Challenge challenge) {
    return Attribution.encode(challenge.realm(), challenge.id());
}
```

TempoChargePayer.java

```
/** Push mode: we broadcast ourselves and hand the courier the transaction hash. */
public String authorization(Challenge challenge, String txHash) {
    String payer = "did:pkh:eip155:" + network.chainId() + ":" + wallet.address();
    return new Credential(challenge.toEcho(),
            Map.of("type", "hash", "hash", txHash), payer).toAuthorization();
}
```

TempoWallet.java

```
// A brand new account's first transfer costs about 545,000 gas on Tempo
// (account creation is charged), so never hard-code a gas limit.
private BigInteger estimateGas(String token, String data) {
    BigInteger estimate = rpc.quantity("eth_estimateGas",
            Map.of("from", address(), "to", token, "data", data));
    return estimate.multiply(BigInteger.valueOf(12)).divide(BigInteger.TEN);
}
```

> Without the challenge-bound memo, the SDK rejects the payment.
>
> Behaviour of mpp-java 0.1.6 seen on testnet, 2026-10-09. Not written in the Tempo charge draft at the time of writing.

06 Break it: five dollars, then paid twice

The first client pays whatever a challenge asks and, on any failure, starts again from the top: a fresh request, a fresh challenge and a fresh payment. Raise the courier's price to 5.00 and it pays 5.00. Drop one paid response and it pays a second time.

MPP tells the client what the server wants to charge. It does not decide what your business is willing to spend.

UnsafePaymentClient.java

```
    public PaidResponse get(URI uri) {
        try {
            return payAndFetch(uri);
        } catch (RuntimeException firstFailure) {
            log.warn("request failed ({}), retrying", firstFailure.getMessage());
            // A fresh request: a fresh challenge, and a fresh payment.
            return payAndFetch(uri);
        }
    }
```

evidence/04-unsafe-overpay.txt

```
$ bin/balance.sh
0x103604c41634224c9914bd79988369d694c69e2a  1999983.548089 pathUSD
0xe614e43e4fbf9867bdf4e2ddbd3f628658b93ddb  16.45 pathUSD


$ curl -s 'http://localhost:18080/break/quotes/premium?from=London&to=Manchester&weightKg=2' |
python3 -m json.tool
{
    "body": "{\"courier\":\"Swift Courier
Premium\",\"price\":8.60,\"currency\":\"GBP\",\"days\":1}",
    "amountPaid": 5,
    "txHash": "0x461659e630c03818bee3ce6b89d46309d718e88d3f1c3c493cb717b9132a7ea2"
}


$ bin/balance.sh
0x103604c41634224c9914bd79988369d694c69e2a  1999978.548048 pathUSD
0xe614e43e4fbf9867bdf4e2ddbd3f628658b93ddb  21.45 pathUSD
```

evidence/05-lost-response.txt (unsafe client)

```
# courier-api log
c.c.c.p.payment.PaymentProblemAdvice     : 402 Payment Required: Payment is required for
"localhost:18090" (Premium courier quotation)., challenge NATM-
jTiYeZ6PMgG5ZSF9rLxMPsoGYU9ffHE2Su64oY
c.c.c.provider.payment.PaymentGate       : payment verified, tx
0x68b0f9aaf4dacb0bf44d432626f03aa52b1cc2b643ce11bdad698acaa7b2ba1e
c.c.c.p.m.LostResponseSimulator          : dropping a paid response on purpose
c.c.c.p.payment.PaymentProblemAdvice     : 402 Payment Required: Payment is required for
"localhost:18090" (Premium courier quotation)., challenge
Aui9gNPm0hqGC6FjzymX8ZZJaWdfhIOUFFqPNdrWSNU
c.c.c.provider.payment.PaymentGate       : payment verified, tx
0x3d543432cb91ecd0cd95ce451c9cfa2aaab4216653267139c0dd7fde427eb85d
```

> Unsafe client: 5.00 paid for one quote at the raised price; 0.10 to the courier, two verified payments, for one quote after a lost response.
>
> Recorded on Tempo Moderato testnet, 2026-10-09: evidence/02 to 05, each produced by the script named on its first line. Test tokens only.

07 Three problems that look like one

Duplicate settlement: money moved twice for one business request. The client's job.

Duplicate redemption: one payment used to buy something else. The server's job; mpp-java's replay store already rejects a reused transaction hash.

Recovery: the payment succeeded, the response was lost, and the customer still needs the answer they paid for. Both sides: a durable operation record on the client, a durable redemption record on the courier.

> The courier never settles a credential twice; it only replays the stored response to the identical request. That is this project's reading of how the core draft's "MUST NOT accept the same credential twice" and its Idempotency-Key guidance fit together, not text from the draft.
>
> A design judgement, stated as such, not a measurement.

08 Fix one: a payment policy before signing

Method, intent, chain, currency, recipient, expiry, then price: at most 0.10 per request and 0.25 per day. An approval writes an APPROVED audit row, and spentToday sums those rows, so approving reserves the budget. approve is synchronized, so inside one process approvals happen one at a time.

synchronized holds within one process only. Across several instances the reservation has to move into the database.

PaymentPolicyService.java

```
/** Synchronized so two payments cannot both squeeze under the daily budget. */
public synchronized PolicyDecision approve(ChargeOffer offer) {
    PolicyDecision decision = check(offer);
    AuditOutcome outcome =
            decision.approved() ? AuditOutcome.APPROVED : AuditOutcome.REFUSED;
    audit.record("safe", outcome, offer, null, decision.reason());
    return decision;
}

private PolicyDecision check(ChargeOffer offer) {
    if (!"tempo".equals(offer.method())) {
        return PolicyDecision.refuse(
                "payment method " + offer.method() + " is not allowed");
    }
    if (!"charge".equals(offer.intent())) {
        return PolicyDecision.refuse(
                "intent " + offer.intent() + " is not allowed");
    }
    if (offer.chainId() != limits.chainId()) {
        return PolicyDecision.refuse("chain " + offer.chainId()
                + " is not the approved chain " + limits.chainId());
    }
    if (!limits.currency().equalsIgnoreCase(offer.currency())) {
        return PolicyDecision.refuse(
                "currency " + offer.currency() + " is not pathUSD");
    }
    if (limits.allowedRecipients().stream()
            .noneMatch(allowed -> allowed.equalsIgnoreCase(offer.recipient()))) {
        return PolicyDecision.refuse(
                "recipient " + offer.recipient() + " is not an approved courier");
    }
    if (offer.expires() == null) {
        return PolicyDecision.refuse("challenge has no expiry");
    }
    if (!offer.expires().isAfter(clock.instant())) {
        return PolicyDecision.refuse("challenge expired at " + offer.expires());
    }
    BigDecimal amount = offer.amount();
    if (amount.compareTo(limits.maxPerRequest()) > 0) {
        return PolicyDecision.refuse(money(amount) + " pathUSD is over the "
                + money(limits.maxPerRequest()) + " per-request limit");
    }
    BigDecimal spentAfter = audit.spentToday().add(amount);
    if (spentAfter.compareTo(limits.dailyBudget()) > 0) {
        return PolicyDecision.refuse(money(amount)
                + " pathUSD would bring today's spend to " + money(spentAfter)
                + ", over the " + money(limits.dailyBudget()) + " daily budget");
    }
    return PolicyDecision.approve();
}
```

retailer-assistant application.properties

```
# The payment authority the assistant has, in pathUSD.
assistant.payment.policy.max-per-request=0.10
assistant.payment.policy.daily-budget=0.25
assistant.payment.policy.allowed-recipients=${COURIER_RECIPIENT_ADDRESS}
assistant.payment.policy.currency=0x20c0000000000000000000000000000000000000
assistant.payment.policy.chain-id=42431
```

PaymentPolicyServiceTest.java

```
    @Test
    void twentySimultaneousRequestsCannotOverspendTheDailyBudget() throws Exception {
        CountDownLatch start = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        try (ExecutorService threads = Executors.newFixedThreadPool(20)) {
            for (int i = 0; i < 20; i++) {
                results.add(threads.submit(() -> {
                    start.await();
                    return policy.approve(offer(TestChallenges.premiumQuote())).approved();
                }));
            }
            start.countDown();
        }

        long approved = 0;
        for (Future<Boolean> result : results) {
            if (result.get()) {
                approved++;
            }
        }
        assertThat(approved).isEqualTo(5);
        assertThat(audit.spentToday()).isEqualByComparingTo("0.25");
    }
```

evidence/04-unsafe-overpay.txt (safe client, price 5.00)

```
$ curl -s 'http://localhost:18080/assistant/quotes/premium?
from=London&to=Manchester&weightKg=2' | python3 -m json.tool
{
    "status": "REFUSED_BY_POLICY",
    "quote": null,
    "quoteFee": 0,
    "paymentTx": null,
    "explorerUrl": null,
    "reason": "5.00 pathUSD is over the 0.10 per-request limit"
}


$ bin/balance.sh
0x103604c41634224c9914bd79988369d694c69e2a  1999978.548048 pathUSD
0xe614e43e4fbf9867bdf4e2ddbd3f628658b93ddb  21.45 pathUSD
```

> Twenty simultaneous approvals: exactly five approved, 0.25 spent. Passing at commit cd2e4d1.
>
> evidence/08-tests.txt (2026-10-09, with testnet): 49 tests, 0 failed. The twenty-thread test was added on 2026-10-10; bin/tests.sh at commit cd2e4d1 runs 44 offline tests, 0 failed, including it.

> A refused payment leaves the payer's nonce at 27 and both balances unchanged; the sixth 0.05 quote of the day is refused at 0.30.
>
> Recorded on Tempo Moderato testnet, 2026-10-09: evidence/02 to 05, each produced by the script named on its first line. Test tokens only.

09 Fix two: recover, never pay again

Before any money moves, the client writes the challenge, the signed transaction and the request key. The hash is known before broadcast. After that it only resumes: PENDING_PAYMENT rebroadcasts the same signed transaction, BROADCAST presents the same credential with the same Idempotency-Key, and if the courier will not hand the result over it marks the operation UNDELIVERED and stops. It never signs a second transfer for that request.

The courier records what each payment bought. The same credential, Idempotency-Key, parcel and challenge get the stored quote and original receipt. Anything else is refused with 402 verification-failed. mpp-java's replay protection stays behind this check.

PaymentOperationStore.java

```
public PaymentOperation startPayment(String requestKey, Challenge challenge,
        SignedTransfer transfer) {
    db.sql("""
            INSERT INTO payment_operation (request_key, state, www_authenticate,
                tx_hash, raw_transaction, updated_at)
            VALUES (?, 'PENDING_PAYMENT', ?, ?, ?, ?)""")
            .params(requestKey, challenge.toWwwAuthenticate(), transfer.hash(),
                    transfer.rawTransaction(), now())
            .update();
    return find(requestKey).orElseThrow();
}
```

MppPaymentClient.java

```
private PaidResponse resume(URI uri, PaymentOperation operation) {
    Challenge challenge = operation.challenge();
    ChargeOffer offer = ChargeOffer.from(challenge);
    String txHash = operation.txHash();
    switch (operation.state()) {
        case REDEEMED -> {
            return new PaidResponse(operation.body(), offer.amount(), txHash);
        }
        case PENDING_PAYMENT -> {
            // Broadcasts the SAME signed transfer; a no-op if it is already mined.
            payer.settle(operation.signedTransfer());
            operations.markBroadcast(operation.requestKey());
            audit.record("safe", AuditOutcome.PAID, offer, txHash,
                    "transfer mined");
        }
        case BROADCAST, UNDELIVERED -> log.info("resuming paid request {}", txHash);
    }
    return redeem(uri, operation, offer);
}
```

RedemptionService.java (courier-api)

```
    public Optional<Redemption> findEarlier(
            String authorization, String idempotencyKey, QuoteRequest request) {
        Optional<Credential> credential = parse(authorization);
        Optional<Redemption> earlier = credential.flatMap(RedemptionService::paidHash)
                .flatMap(store::findByTxHash);
        if (earlier.isEmpty()) {
            return Optional.empty();
        }
        Redemption redemption = earlier.get();
        boolean sameRequest = idempotencyKey != null
                && idempotencyKey.equals(redemption.idempotencyKey())
                && redemption.requestFingerprint().equals(request.fingerprint())
                && redemption.challengeId().equals(credential.get().challenge().id());
        if (!sameRequest) {
            throw new VerificationFailedException(
                    "transaction hash already redeemed: " + redemption.txHash());
        }
        log.info("recovered paid quote for tx {}", redemption.txHash());
        return earlier;
    }
```

evidence/05-lost-response.txt (safe client)

```
# courier-api log
c.c.c.p.payment.PaymentProblemAdvice     : 402 Payment Required: Payment is required for
"localhost:18090" (Premium courier quotation)., challenge
4mrdZvdbtGICwTRZGV9SYFfDvnTuRfIIcNVFPeGJzBc
c.c.c.provider.payment.PaymentGate       : payment verified, tx
0xf370f84212087d72ec7239eaab25c6b32f37820e5a64a0fb8bcfac9e9432226d
c.c.c.p.m.LostResponseSimulator          : dropping a paid response on purpose
c.c.c.p.redemption.RedemptionService     : recovered paid quote for tx
0xf370f84212087d72ec7239eaab25c6b32f37820e5a64a0fb8bcfac9e9432226d
```

> Safe client, same lost response: courier +0.05, one transfer, audit APPROVED, PAID, RESPONSE_LOST, DELIVERED. A redeemed hash presented for a 9 kg parcel: 402 verification-failed.
>
> Recorded on Tempo Moderato testnet, 2026-10-09: evidence/01-http-exchange.txt and 06-replay-rejected.txt, written by HttpExchangeTranscriptTestnetIT. Test tokens only.

10 The transactions in the video

Payer 0x103604c41634224c9914bd79988369d694c69e2a, courier 0xe614e43e4fbf9867bdf4e2ddbd3f628658b93ddb, both public testnet addresses. Open any hash at explore.testnet.tempo.xyz/tx/<hash>.

Safe client, one paid quote, 0.05:
0xc6dcf417344f9a3c6535f76fba782d9b8f45ae5644744d50c354ef8f5e8027d3

Claude Code compares the couriers, 0.05:
0x97a72969802b50522d5fe4eb5a5622fa3b6a56967fb3fd80412df08fe47c3817

Unsafe client, price raised, 5.00:
0x84feb59189a69135aeec3b4ad5e29567e58df94d20e73be44eb731f98e0c530d

Unsafe client, lost response, first payment:
0xceb676cee429080efaebd8f32d0cad66570787a312b65fdf101b3a4fdf9cbaf7

Unsafe client, lost response, second payment:
0x72108c53cda1cb16d585f33007501caaa2db3250b7a3828b242f78356eb1c2a6

Safe client, lost response, one payment:
0x70e97ab959bde6e0e70e2f396254df204ae8a45966b48f3351bc20a274f4b3fe

Claude Code final run, 5.00 refused first, then 0.05:
0xb4456e6b636b5eb7ef82ee4e6ef49a8931073e52df36b337d20d9b1d77aa6e92

> Seven testnet transactions on screen in the episode.
>
> `The runs recorded for the episode, 2026-10-11, Tempo Moderato testnet. Open any hash at explore.testnet.tempo.xyz/tx/<hash>.`

> mpp-java 0.1.6 experimental; MPP an Internet-Draft; chain 42431; pathUSD 6 decimals.
>
> `At the time of writing, October 2026: com.stripe:mpp-java 0.1.6 (its README: experimental, API subject to change); MPP an Internet-Draft by Stripe and Tempo, living text at paymentauth.org; Tempo Moderato chain 42431; pathUSD with 6 decimals.`

Before you let software spend money

1. Treat a 402 as an offer: read every field of the challenge before deciding anything
2. Check method, network, currency, recipient and expiry, not only the price
3. Enforce a per-request limit and a daily budget in code, before signing
4. Reserve the budget at approval, and make approvals atomic
5. Never rely on the AI calling the tool to decide its own limits
6. Bind the payment to the challenge (the memo), or the server will reject it
7. Estimate gas every time; a new wallet's first transfer costs far more
8. Write the signed transaction down before broadcasting; after that, only resume
9. On a lost response, present the same credential and Idempotency-Key, never a new payment
10. If the result cannot be recovered, stop and report the hash; do not pay again
11. Check the receipt's reference is your own transaction hash
12. Never log a credential: it is a bearer token. Log the hash
13. Use TLS for anything beyond localhost

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The repository: courier-api, retailer-assistant, tempo-wallet, the tests and every evidence file
https://github.com/code-with-sam-dev/spring-boot-mpp-courier

MPP specification (living text)
https://paymentauth.org

MPP documentation
https://mpp.dev

Model Context Protocol
https://modelcontextprotocol.io

Tempo testnet explorer
https://explore.testnet.tempo.xyz

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Spring and Spring Boot are trademarks of Broadcom. Java is a trademark of Oracle. Stripe, Tempo, Claude and Claude Code are trademarks of their respective owners. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here. Testnet tokens only; nothing here is financial advice.

Technical claims verified 2026-10-11 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.